

Deep Learning

Chapter 2: Optimizing Deep Neural Networks

Complete Explanatory Study Guide

Prepared for Exam Revision

Topics Covered:

- Gradient Descent Variants
- Momentum & Learning Rate Decay
- RMSProp & Adam Optimizers
- Hyperparameter Tuning
- Overfitting & Regularization
- Dropout & Batch Normalization
- Gradient Checking
- Grid, Random & Bayesian Search

Dr. N. DIF — ESI-SBA

Contents

1	Preliminary Concepts: Batch vs Epoch	2
2	Optimization Algorithms	2
2.1	1.1 What is Optimization?	2
2.2	1.2 Three Types of Gradient Descent	3
	2.2.1 Exercise Walkthrough (Slide 6)	3
2.3	1.3 Gradient Descent with Momentum	4
2.4	1.4 Learning Rate Decay	5
2.5	1.5 RMSProp Optimizer	5
2.6	1.6 Adam Optimizer	6
3	Overfitting, Regularization, and Gradient Checking	7
3.1	2.1 The Three Scenarios: Underfit, Fit, Overfit	7
3.2	2.2 Regularization Techniques	7
	3.2.1 L2 Regularization (Weight Decay)	7
	3.2.2 L1 Regularization	8
	3.2.3 Dropout Regularization	8
	3.2.4 Early Stopping	8
	3.2.5 Batch Normalization	9
3.3	2.3 Gradient Checking	10
4	Hyperparameter Tuning	11
4.1	3.1 What are Hyperparameters?	11
4.2	3.2 Hyperparameter Search Methods	11
	4.2.1 Manual Search (Babysitting)	11
	4.2.2 Grid Search	11
	4.2.3 Random Search	12
	4.2.4 Bayesian Optimization	12
5	Quick Reference Cheat Sheet	14

Preliminary Concepts: Batch vs Epoch

Before diving into optimization algorithms, let's clarify two fundamental concepts that are used everywhere in training.

Key Definitions

Batch Size (n_{batch}): The number of training samples processed *before* the model's weights are updated. Think of it as: "how many examples does the model look at before it corrects itself?"

Epoch (n_{epoch}): One complete pass through the *entire* training dataset. After one epoch, every training example has been seen exactly once.

Number of Iterations: The total number of weight updates performed:

$$n_{iterations} = n_{epoch} \times \frac{N}{n_{batch}}$$

where N is the total number of training samples, and $\frac{N}{n_{batch}}$ is the number of batches per epoch.

Concrete Example

Suppose you have $N = 1000$ training examples, $n_{batch} = 100$, and $n_{epoch} = 10$.

- Each epoch contains $\frac{1000}{100} = 10$ batches (updates).
- Total iterations = $10 \times 10 = 100$ weight updates.

Why randomize data? Because if data is ordered (e.g., all cats first, then all dogs), each batch will be biased. Shuffling ensures each batch is representative of the whole dataset, leading to better and more stable training.

Optimization Algorithms

1.1 What is Optimization?

Optimization in Deep Learning

Optimization is the process of finding the model parameters $W \in \mathbb{R}^d$ that **minimize the loss/cost function** $J(W)$.

If the function is convex: there is a single global minimum — guaranteed to be found.

In deep learning (non-convex): there can be many *local minima*. We aim to find a *good* local minimum or the global one.

The algorithm used for this is called **Gradient Descent**: we iteratively move in the direction that reduces the loss.

1.2 Three Types of Gradient Descent

Gradient Descent

An iterative algorithm that adjusts parameters by moving in the direction of the negative gradient (the steepest descent):

$$W \leftarrow W - \eta \cdot \nabla_W J(W)$$

where η is the **learning rate** (step size).

Type	Description	Update Rule
Batch GD	Uses all N examples per update. Very stable, slow.	$n_{batch} = N$
Stochastic GD (SGD)	Uses one example per update. Very fast, very noisy.	$n_{iterations} = n_{epoch} \times N$
Mini-Batch GD	Uses n_{batch} examples per update. Best balance.	$n_{iterations} = n_{epoch} \times \frac{N}{n_{batch}}$

Advantages & Disadvantages

Type	Advantages	Disadvantages
Batch	Smooth convergence, unbiased gradient, guaranteed to converge on convex loss	Slow on large datasets; memory issues
Mini-Batch	Faster than batch; noise helps avoid local minima; best in practice	Requires learning rate decay due to oscillations
Stochastic	Same as mini-batch	Most noisy; hardest to converge

The Oscillation Problem

During optimization, the search can oscillate a lot on the **vertical axis** (wrong direction) while being slow on the **horizontal axis** (direction toward the minimum). This prevents using large learning rates and slows convergence. Solutions: **Momentum**, **RMSProp**, **Adam**.

Exercise Walkthrough (Slide 6)

Given: $h(x) = w^T x$, initial weights $w_0 = 0, w_1 = 1, w_2 = 0.5$, and training data:

$$X = \begin{bmatrix} 1 & 2 \\ -2 & 5 \\ 0 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 6 \\ 1 \end{bmatrix}$$

Part a: Forward pass for x

T_1

$$h(x) = w_0 \cdot 1 + w_1 \cdot 2 + w_2 \cdot 3 = 0 \cdot 1 + 1 \cdot 2 + 0.5 \cdot 3 = 2 + 1.5 = \mathbf{3.5}$$

Part b: Loss Calculation

$$L(w) = \frac{1}{2} \sum_i (y_i - w^T x_i)^2$$

Predictions: $\hat{y}_1 = 0 + 1 + 1 = 2$, $\hat{y}_2 = 0 - 2 + 2.5 = 0.5$, $\hat{y}_3 = 0 + 0 + 0.5 = 0.5$

$$L = \frac{1}{2} [(1 - 2)^2 + (6 - 0.5)^2 + (1 - 0.5)^2] = \frac{1}{2} [1 + 30.25 + 0.25] = \mathbf{15.75}$$

Part c: Batch Gradient Descent Update (η)

Gradient: $\nabla_w L = -\sum_i (y_i - \hat{y}_i) x_i$

Errors: $e_1 = 1 - 2 = -1$, $e_2 = 6 - 0.5 = 5.5$, $e_3 = 1 - 0.5 = 0.5$

$$\nabla_{w_0} = -(-1)(1) - (5.5)(-2) - (0.5)(0) = 1 + 11 = 12$$

$$\nabla_{w_1} = -(-1)(1) - (5.5)(5) - (0.5)(1) = 1 - 27.5 - 0.5 = -27 \Rightarrow w_1^{new} = 1 - 0.01 \cdot (-27) = 1.27$$

The batch GD uses *all* three examples to compute a single gradient, then updates once.

1.3 Gradient Descent with Momentum

Momentum

Momentum is an extension of gradient descent that adds a **velocity** term. Instead of moving in the direction of the current gradient only, it also considers the direction of previous updates. This *damps oscillations* and accelerates convergence.

Momentum Update Equations

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \Delta W_t$$

$$W_{t+1} = W_t - \eta \cdot V_t$$

- V_t : velocity (exponentially weighted average of past gradients)
- β : decay factor (usually **0.9**) — controls how much past velocity matters
- ΔW_t : current gradient ($\nabla_w J$)

Intuition: Imagine a ball rolling down a hill. It doesn't just respond to the current slope — it carries momentum from past movement, helping it roll past small bumps (local minima) and reducing zigzagging.

1.4 Learning Rate Decay

Learning Rate Decay

Start training with a **large learning rate** (to move quickly away from bad regions), then **gradually reduce it** so the model can settle precisely into the minimum.

Three Decay Formulas

Time-based decay:

$$LR = LR_0 \times \frac{1}{1 + \text{decay_rate} \times \text{epoch_num}}$$

Step-based decay:

$$LR = LR_0 \times \text{Drop_rate}^{\lfloor \text{epoch_num} / \text{epoch_drop} \rfloor}$$

Exponential decay:

$$LR = LR_0 \times e^{-k \cdot t}$$

where k is the decay rate and t is the epoch number.

Why reduce the learning rate?

- **At the beginning:** large LR to escape high-error zones quickly.
- **At the end:** small LR so the model settles precisely into the deepest minimum without bouncing around.

1.5 RMSProp Optimizer

RMSProp (Root Mean Squared Propagation)

RMSProp is an **adaptive** optimizer: each parameter gets its own learning rate. It normalizes the gradient by a moving average of squared past gradients, which reduces oscillations.

RMSProp Update Equations

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot (\Delta W_t)^2$$

$$W_{t+1} = W_t - \eta \cdot \frac{\Delta W_t}{\sqrt{V_t + \epsilon}}$$

- $\beta = 0.999$ (typically)
- $\epsilon = 10^{-8}$ (small constant to avoid division by zero)

Key insight: If a parameter has been changing a lot (large V_t), its effective learning rate is *reduced* (divided by large number). If it hasn't changed much (small V_t), it gets a *larger* effective step. This is the “per-parameter learning rate” idea.

1.6 Adam Optimizer

Adam (Adaptive Moment Estimation)

Adam combines **Momentum** (remembers direction of past gradients) with **RMSProp** (normalizes by squared gradients). It is the most widely used optimizer in practice.

Adam Update Equations

Momentum term (1st moment):

$$V_t = \beta_1 \cdot V_{t-1} + (1 - \beta_1) \cdot \Delta W_t$$

RMSProp term (2nd moment):

$$S_t = \beta_2 \cdot S_{t-1} + (1 - \beta_2) \cdot (\Delta W_t)^2$$

Weight update:

$$W_{t+1} = W_t - \eta \cdot \frac{V_t}{\sqrt{S_t + \epsilon}}$$

Typical values: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Summary: How Adam thinks

- Momentum (V_t): “What direction have I been moving in recently?”
- RMSProp (S_t): “How much has this parameter been changing? Scale accordingly.”
- Result: Fast, smooth convergence with minimal oscillation, automatically adapts to each weight.

Optimizer Comparison Summary

Optimizer	Core Idea	Best Used When
SGD	Pure gradient descent	Simple problems, theory
SGD + Momentum	Add inertia to reduce oscillation	Better than plain SGD
RMSProp	Per-parameter adaptive LR	Non-stationary problems (RNN)
Adam	Momentum + RMSProp	General default choice

Overfitting, Regularization, and Gradient Checking

2.1 The Three Scenarios: Underfit, Fit, Overfit

Three Model States

Underfitting: Model is too simple. It performs *poorly* on both training and test data. The model has not learned the problem. (High bias)

Good Fit: Model learns the training data well *and* generalizes well to unseen test data. This is the goal!

Overfitting: Model learns the training data *too well* — it memorizes noise. Performs great on training, *poorly* on test data. (High variance)

Visual intuition:

- As model complexity increases: training error keeps decreasing.
- But test error first decreases (fitting), then *increases* (overfitting).
- The sweet spot between these is the “best fit”.

2.2 Regularization Techniques

Regularization = techniques to **prevent overfitting** by discouraging the model from becoming too complex.

L2 Regularization (Weight Decay)

L2 Regularization

Add the squared magnitude of all weights to the loss function:

$$C(w, b) = \frac{1}{m} \sum_{i=1}^n L(\hat{y}_i, y_i) + \frac{\lambda}{2m} \sum_{l=1}^L \|W^{[l]}\|^2$$

where λ is the **regularization parameter** (e.g., 0.01 to 0.4).

The weight update becomes:

$$W \leftarrow W - \alpha \left[\frac{\partial J}{\partial W} + \frac{\lambda}{m} W \right] = \underbrace{\left(1 - \frac{\alpha \lambda}{m} \right)}_{\text{decay factor}} W - \alpha \frac{\partial J}{\partial W}$$

Why does shrinking weights prevent overfitting? Smaller weights make each neuron’s contribution less extreme. The network becomes less sensitive to individual training examples, so it learns general patterns rather than memorizing noise.

L1 Regularization

L1 Regularization

Add the **absolute values** (not squared) of all weights to the loss:

$$C(w, b) = \frac{1}{m} \sum_{i=1}^n L(\hat{y}_i, y_i) + \frac{\lambda}{2m} \sum_{l=1}^L \|W^{[l]}\|$$

L1 vs L2 Key Difference

- **L2:** Weights shrink toward 0 but rarely become *exactly* 0. All neurons remain active but with smaller influence.
- **L1:** Can push weights to *exactly* 0 — effectively **removing** neurons entirely. Creates sparse models.
- In practice: **L2 is used more often** in deep learning; L1 is used when you want feature selection.

Dropout Regularization

Dropout

During *each* training pass, randomly **deactivate** (set to 0) a fraction of neurons in each layer, according to a **dropout rate** (e.g., 0.5 = 50% of neurons dropped).

Why does dropout work?

- Deep networks are prone to overfitting due to the large number of parameters.
- By removing random neurons each pass, the network cannot rely on any single neuron — every neuron must learn robust, general features.
- It effectively trains many different “sub-networks” and averages their results (similar to ensemble learning).

Important: Dropout is **ONLY** used during training!

During **testing/inference**, dropout is **disabled**. All neurons are active. This ensures consistent, deterministic predictions. The weights are typically scaled to compensate.

Early Stopping

Early Stopping

Monitor the model’s performance on a **validation set** after each epoch. Stop training when validation performance *starts to degrade*, even if training loss is still improving.

Three elements of early stopping:

1. **Monitor:** Choose a validation set and metric (e.g., validation loss or accuracy).
2. **Trigger:** Stop when validation loss increases over k consecutive epochs (patience).
3. **Choose model:** Save the model weights from the *best* epoch (best validation performance).

Intuition: At some point, the training loss keeps decreasing, but the model is just memorizing the training data. The validation error starts rising. Early stopping says: “stop here, before overfitting gets worse.”

Batch Normalization

Batch Normalization

An extra layer added to a neural network that **normalizes** the inputs to each layer (zero mean, unit variance) before passing them to the next layer. It operates on mini-batches, not single samples.

Batch Normalization Algorithm

For a mini-batch $\mathcal{B} = \{x_1, \dots, x_m\}$:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad y_i = \gamma \hat{x}_i + \beta$$

where γ (scale) and β (shift) are **learnable parameters** (updated via backpropagation).

Benefits:

- **Speeds up training:** Normalized inputs are easier to optimize.
- **Handles internal covariate shift:** When the distribution of layer inputs shifts during training (because earlier layers keep changing), BN re-normalizes them, stabilizing training.
- Acts as a mild regularizer.

What is Internal Covariate Shift?

As earlier layers update during training, the distribution of their outputs changes. This means later layers constantly receive inputs with shifting statistics — making them harder to train. Batch Normalization fixes this by normalizing at each layer, keeping distributions stable.

2.3 Gradient Checking

Gradient Checking

A **debugging technique** to verify that your backpropagation implementation is correct. It compares the analytically computed gradient with a *numerically approximated* gradient.

Numerical Gradient Approximation

For each parameter θ_i , approximate the derivative using the centered difference formula:

$$\frac{dJ}{d\theta_i} \approx \frac{J(\theta + \epsilon) - J(\theta - \epsilon)}{2\epsilon}$$

where $\epsilon \approx 10^{-4}$ (small constant).

When to use Gradient Checking

- Use it **only during debugging**, not during actual training (it is extremely slow).
- If the numerical and analytical gradients differ significantly, there is a bug in your backpropagation code.

Hyperparameter Tuning

3.1 What are Hyperparameters?

Hyperparameters vs Parameters

- **Model Parameters** (W, b): Learned automatically during training.
- **Hyperparameters**: Set by the engineer *before* training. Cannot be learned from data directly.

Two Categories of Hyperparameters

Network Structure:

- Number of hidden layers and units per layer
- Dropout rate (typical: 0.2, 0.4, 0.5)
- Weight initialization method
- Activation function (ReLU, sigmoid, tanh, ...)

Training Algorithm:

- Learning rate (typical: 0.1, 0.01, 0.001)
- Momentum / decay factor β
- Number of epochs (20, 50, 100, ...)
- Batch size (32, 64, 128, 256, ...)
- Optimizer (SGD, Momentum, RMSProp, Adam)

3.2 Hyperparameter Search Methods

Manual Search (Babysitting)

The researcher manually tries different values based on intuition and experience. Good for experts, but not systematic.

Grid Search

Grid Search

Define a grid of all possible combinations of hyperparameter values, train a model for *every* combination, and pick the best.

Grid Search Example

If you search over: $\text{learning_rate} \in \{0.1, 0.01, 0.001\}$, $\text{batch_size} \in \{32, 64, 128\}$
 You must train $3 \times 3 = 9$ models and evaluate each. For 5 hyperparameters with 5 values each: $5^5 = 3125$ models!

Advantages

Guaranteed to find the best combination in the search space

Disadvantages

Curse of dimensionality: exponential growth of combinations
 Very computationally expensive

Random Search

Random Search

Instead of evaluating *all* grid points, randomly sample hyperparameter combinations from the search space. Evaluate a fixed number of combinations.

Advantages

Efficient on high-dimensional spaces
 Gives good results in fewer iterations

Disadvantages

Does NOT guarantee finding the best combination

Shared Weakness of Grid and Random Search

Each new trial is independent of all previous trials! Neither method learns from past results to decide where to search next. This is inefficient.

Bayesian Optimization

Bayesian Optimization

Builds a **probabilistic model** (surrogate model) of the objective function based on past evaluations. Uses this model to intelligently decide *where to search next*, focusing on promising regions.

Bayesian optimization addresses the key weakness of grid/random search by **learning from previous results**, making it the most sample-efficient method for expensive hyperparameter searches.

Overall Comparison			
Method	Guarantees Best?	Cost	Learns from History?
Grid Search	Yes (in grid)	Very High	No
Random Search	No	Medium	No
Bayesian Optimization	No	Low	Yes

Quick Reference Cheat Sheet

Key Formulas to Memorize

Gradient Descent:

$$W \leftarrow W - \eta \cdot \nabla_W J$$

Momentum:

$$V_t = \beta V_{t-1} + (1 - \beta) \Delta W_t, \quad W_{t+1} = W_t - \eta V_t$$

RMSProp:

$$V_t = \beta V_{t-1} + (1 - \beta) (\Delta W_t)^2, \quad W_{t+1} = W_t - \eta \frac{\Delta W_t}{\sqrt{V_t + \epsilon}}$$

Adam:

$$\begin{aligned} V_t &= \beta_1 V_{t-1} + (1 - \beta_1) \Delta W_t \\ S_t &= \beta_2 S_{t-1} + (1 - \beta_2) (\Delta W_t)^2 \\ W_{t+1} &= W_t - \eta \frac{V_t}{\sqrt{S_t + \epsilon}} \end{aligned}$$

L2 Regularization:

$$C = \frac{1}{m} \sum L(\hat{y}_i, y_i) + \frac{\lambda}{2m} \sum_l \|W^{[l]}\|^2$$

Gradient Checking:

$$\frac{dJ}{d\theta} \approx \frac{J(\theta + \epsilon) - J(\theta - \epsilon)}{2\epsilon}, \quad \epsilon = 10^{-4}$$

Batch Normalization:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

Time-based LR Decay:

$$LR = LR_0 \times \frac{1}{1 + \text{decay_rate} \times \text{epoch}}$$

Typical Hyperparameter Values

Hyperparameter	Typical Values
Learning rate η	0.1, 0.01, 0.001
Momentum β	0.9
RMSProp β	0.999
Adam β_1	0.9
Adam β_2	0.999
ϵ (numerical stability)	10^{-8}
Dropout rate	0.2, 0.4, 0.5
Regularization λ	0.01, 0.02, ..., 0.4
Batch size	32, 64, 128, 256

Good luck on your exam!

Remember: Understanding the *intuition* behind each concept is more important than memorizing formulas. Think about what each optimizer is trying to fix, and why each regularization technique helps.